

A3D: Agentic AI flow for autonomous Accelerator Design

Abinand Nallathambi
Purdue University
West Lafayette, IN, USA

Christopher Knight
Argonne National Laboratory
Argonne, IL, USA

Shantanu Ganguly
Unaffiliated

Wilfried Haensch
University of Chicago
Chicago, IL, USA

Anand Raghunathan
Purdue University
West Lafayette, IN, USA

Abstract

Accelerating applications through the design of hardware accelerators can significantly enhance system performance and energy efficiency. Despite advances, such as high-level synthesis (HLS), designing accelerators for complex applications still remains highly labor-intensive, demanding considerable expertise in understanding workloads to be accelerated, hardware design, micro-architecture, and EDA tool usage, posing challenges for application domain experts. Therefore, most accelerator solutions are limited to applications with a regular predictable dataflow. Advances in AI have enabled agents that perform autonomous planning, reasoning, execution and reflection, leading to unprecedented potential for automation through agentic AI. We present A3D, an Agentic AI flow for end-to-end Automation of hardware Accelerator Design. A3D automates workload analysis, performance bottleneck identification, code refactoring for HLS compatibility and micro-architecture generation. A3D also generates diverse accelerator designs by automatically exploring the speed-area tradeoff space. Recent efforts have explored the use of AI for specific tasks such as design space exploration in HLS, leaving several tasks to still be performed manually. A3D addresses the challenges in applying modern LLMs to accelerator design by judiciously partitioning tasks among specialist agents, orchestrating process loops with specialist and verifier agents, utilizing pre-existing and custom tools, and employing agentic RAG for codebase and proprietary EDA tool documentation exploration. Our implementation of A3D, using commercial components like Claude Sonnet 4.5 and the Catapult HLS tool, demonstrates its effectiveness by generating accelerator designs with no human intervention from complex scientific applications like LAMMPS (molecular dynamics simulation) and QMCPACK (quantum chemistry).

CCS Concepts

• **Software** → **Hardware tools for EDA.**

Keywords

AI agents, EDA, Accelerator Design, High-Level Synthesis, Scientific computing

1 Introduction

Specialization, or the design and use of hardware accelerators, is a fundamental technique in computing system design and has the potential to speed up the execution of applications by orders of magnitude [4, 8]. Despite the well-known potential of hardware

accelerators, their use in practice tends to be limited to a few application domains (ML [3, 12], cryptography [24], audio/video codecs) where computational kernels can be easily re-used across a wide range of workloads. The high levels of manual effort involved in designing hardware accelerators have been a significant impediment to their broader use. Advances in abstraction and automation such as high-level synthesis (HLS) [15, 17] have addressed this challenge in part by enabling software-like specifications to be converted into register-transfer level (RTL) implementations. Nevertheless, designing a hardware accelerator for complex application workloads still requires significant effort as the designer needs to perform various steps such as profiling the application workload to identify target kernels for acceleration, re-writing the code to separate out the identified kernels, ensuring that the kernels adhere to the subset of programming constructs and data types supported by the HLS tool, and running the HLS tool to generate the RTL implementation. Notably, these steps require designers with considerable expertise in hardware design, micro-architecture and HLS tool use, making them challenging for application domain experts and software developers. Recent advances in AI have led to AI agents that can autonomously set goals, plan and execute them with minimal or no human intervention [34]. These advances have enabled unprecedented levels of automation in a wide range of tasks, from generating natural language[36], images[2] and video[6, 27] to code generation[10] and even scientific research[18]. We explore the use of agentic AI to realize end-to-end automation of all the steps involved in hardware accelerator design. Realizing this vision presents significant challenges. Despite recent advances in agentic and reasoning capabilities of AI systems, the underlying large language models (LLMs) are still prone to a wide range of inadequacies such as hallucination [9], laziness and context rot[1]. Furthermore, since LLMs are trained on public data from the Internet, they are typically not exposed to the capabilities and usage of commercial EDA tools. Given the complexity of the task at hand, architecting the agentic flow in a manner that is cognizant of the limitations of artificial intelligence systems and the potential pitfalls they could face is vital to achieve reliable end-to-end automation of the accelerator design flow. To that end, we present A3D, an agentic AI flow for end-to-end automated accelerator design. Starting with the codebase for an application (C/C++/CUDA), A3D executes and analyzes the workload, determines kernels that are performance bottlenecks, re-factors the code to extract the identified bottleneck kernels, prepares the kernels to be HLS-ready, and runs the HLS tool to generate the micro-architectural design (register-transfer

Framework	Application Analysis	Workload Profiling	Kernel Extraction	Code Refactoring	Numerics Optimization	HLS Synthesis	Design Space Exploration	GPU Workloads
C2HLSC [5]	–	–	–	✓	–	✓	–	–
SAGE-HLS [14]	–	–	–	✓	–	–	–	–
ChatHLS [16]	–	–	–	–	–	✓	✓	–
LIFT [20]	–	–	–	–	–	–	✓	–
LLM-DSE [29]	–	–	–	–	–	–	✓	–
HLSPilot [31]	–	–	–	✓	–	✓	✓	–
HLS-Repair [32]	–	–	–	✓	✓	✓	–	–
HLSRewriter [33]	–	–	–	✓	–	✓	–	–
Agentic-HLS [19]	–	–	–	–	–	–	✓	–
A3D (This work)	✓	✓	✓	✓	✓	✓	✓	✓

Table 1: Coverage of HLS workflow stages across related frameworks. Existing work addresses individual downstream stages; A3D is the first to automate the full pipeline from application analysis through design space exploration.

level) for diverse accelerator designs spanning the speed-area trade-off space. To address the aforementioned challenges, the agentic architecture of A3D has the following key features:

- A judicious partitioning of the tasks involved in end-to-end accelerator design (workload analysis, profiling, refactoring for HLS compatibility, etc.) into a set of steps with a dedicated (specialist) agent assigned to each step.
- The agents employ pre-existing tools as well as custom tools that we implemented using the model context protocol (MCP) to strike a balance between using generative capabilities of the underlying LLM and the reliable functionality of deterministic tools. This careful delineation boosts the capabilities of the agents beyond what an LLM could achieve.
- Furthermore, each specialist agent is overseen by a dedicated verifier agent that iterates until the specialist agent completes the tasks correctly.
- A3D uses agentic RAG with a two-stage retrieval pipeline using state-of-the-art embedding and reranking models to explore the codebase and proprietary EDA tool documentation.

2 Related Works

Recent works have developed specialized LLM-driven frameworks targeting individual stages of the HLS-based accelerator design workflow. We organize them by the stage they address and then identify the gap that A3D fills.

Code refactoring for synthesizability. Translating standard C/C++ into the synthesizable subset accepted by HLS tools is a labor-intensive prerequisite that several frameworks now automate. SAGE-HLS [14] fine-tunes a specialized model on a novel Verilog-to-C dataset using an AST-guided training objective, enabling the model to learn synthesis-friendly code structure rather than surface-level text patterns. C2HLSC [5] takes a tool-in-the-loop approach: it iteratively submits refactored code to the HLS compiler, parses error logs, and re-prompts the LLM until synthesis succeeds. HLS-Repair [32] similarly closes the loop with the HLS tool but additionally addresses numeric type optimization, converting floating-point

operations to fixed-point representations to improve hardware efficiency. HLSRewriter [33] extends C2HLSC’s iterative approach with a multi-pass refactoring pipeline that handles a broader set of unsupported constructs, including pointer arithmetic and dynamic memory, achieving higher synthesis success rates. HLSDebugger [30] targets the downstream problem of identifying and correcting logic bugs in HLS code using LLM-based solutions.

Design space exploration. Once a kernel is synthesizable, selecting the right combination of optimization directives (loop unrolling factors, pipeline initiation intervals, array partitioning schemes) remains an expert-driven art due to the large design space [25]. LIFT [20] tackles this with a hybrid LLM–GNN architecture: a Graph Neural Network reasons over control and data dependencies to supervise the LLM’s fine-tuning for pragma insertion, substantially outperforming both heuristic-based DSE tools and naïve general-purpose LLM insertions. LLM-DSE [29] deploys a multi-agent team that collaboratively navigates the design space; the Critic agent parses HLS synthesis reports and provides natural-language feedback to guide subsequent iterations. ChatHLS [16] separates concerns into an HLSFixer agent for automated error correction and an HLSTuner agent that iteratively refines pragma configurations through synthesis-report feedback. Agentic-HLS [19] combines an LLM with a GNN in a chain-of-thought reasoning framework to predict HLS quality-of-result metrics before synthesis, enabling rapid design space pruning.

End-to-end orchestration. HLSPilot [31] is the closest prior art to A3D, orchestrating profiling, C-to-HLS translation, DSE, and host-side code generation in a four-stage pipeline augmented with retrieval-augmented generation from Xilinx Vitis documentation. However, HLSPilot, and all frameworks above, share a critical assumption: they begin with isolated, well-defined kernel functions where the performance bottleneck has already been extracted into a standalone test harness and are not designed to handle the complexity of full application analysis, kernel identification and extraction.

The tasks that *precede* this starting point remain unaddressed in all existing frameworks: analyzing HPC-scale applications with hundreds of source files, profiling representative workloads, identifying bottleneck kernels buried within deep class hierarchies, resolving

their data-structure dependencies across multiple abstraction layers, and isolating them into self-contained harnesses. Table 1 summarizes this coverage gap. A3D closes this gap by automating these aforementioned upstream tasks along with the downstream tasks like refactoring, HLS synthesis and design space exploration, thus achieving end-to-end automation of going from application to accelerator. Additionally, most scientific applications are accelerated on GPUs despite being poorly served by the programmable parallelism offered by GPUs and could benefit from custom accelerators. None of the existing frameworks address such scenarios, where performance bottlenecks reside in CUDA kernels. A3D is capable of handling CUDA bottlenecks and developing custom accelerator designs directly from CUDA code.

3 Agentic AI flow for Accelerator Design

In this section, we present A3D, an Agentic AI flow for Automated Accelerator Design. We present the technical details and design choices behind A3D, and how it leverages the agentic capabilities of LLMs while addressing their limitations.

The A3D flow starts with the application codebase and a workload to execute on the application, along with general natural language instructions regarding compilation, execution and environment setup. An *application* is typically a software program that was designed to run on general-purpose computing platforms such as CPUs and GPUs. The application might contain thousands of source files typically written in a high-level programming language such as C/C++, CUDA or Python to meet the functional and performance requirements of the target domain. In this work, we focus on scientific computing applications written in C/C++ for molecular dynamics simulations and quantum chemistry. A *workload* is an input configuration that exercises the application to perform a specific task or calculation. A workload typically contains input data files and requirements for the application to run. The workloads can vary significantly in terms of data size, complexity, and computational requirements and the workloads studied in this work are typically run on high-performance computing (HPC) systems.

A3D decomposes the complex process of accelerator design into three major phases, as illustrated in Figure 1: (1) the Analysis phase, where the application and workload are analyzed to identify performance bottlenecks and the dependencies of the bottlenecks are determined, (2) the Preparation phase, where the identified bottlenecks are isolated from the application codebase and refactored to be compatible with HLS tools, and (3) the Synthesis phase, where the prepared bottlenecks are synthesized using an HLS tool to generate hardware accelerator designs. Each phase is further decomposed into smaller manageable tasks that are assigned to specialist agents designed to perform these tasks effectively. The cross-cutting infrastructure principles of tool augmentation, agentic RAG, adversarial verification, and orchestration are described in §3.4.

3.1 Analysis Phase

The algorithmic and computational complexity of scientific computing applications often lead to performance bottlenecks that limit their overall performance on programmable general-purpose computing platforms. These bottlenecks can arise from various factors

such as complexity of algorithms, suboptimal data structures, memory access patterns, and unsuitability for the traditional SIMD or SMT parallelism paradigms employed by CPUs and GPUs. Identifying these bottlenecks and building accelerators to offload them can significantly improve the overall performance of the application. However, identifying these bottlenecks in large complex applications is a challenging task that requires expertise in both the application domain and software development, as the bottlenecks can be deeply embedded within the application codebase with complex dependencies.

A3D addresses this challenge with three specialist agents that work in sequence to identify the bottlenecks effectively. The first agent, the *Workload Analyst*, analyzes the application codebase using semantic search and the workload to develop an understanding of the application’s functionality and the workload’s characteristics. The second agent, the *Performance Profiler*, uses a profiling tool like gprof to profile the application with the given workload and identify the functions that consume the most execution time. The third agent, the *Bottleneck Analyst*, builds on the insights from the previous two agents to analyze the identified performance-critical functions and identify their dependencies within the application codebase. Determining the list of data structure dependencies is critical for accurate implementation of the bottleneck kernels. In order to ensure that the Bottleneck Analyst can reliably and repeatedly produce the exhaustive list of dependencies, a static analysis tool is implemented using CodeQL. The Bottleneck Analyst uses said codeQLAnalysis tool to perform static analysis of the codebase and identify the data structure and function dependencies of the performance-critical functions. The Bottleneck Analyst then produces a detailed report that summarizes the identified bottlenecks and provide valuable insights into the data types, sizes, and structures used by these performance-critical functions.

3.2 Preparation Phase

High level synthesis is a widely used methodology for designing hardware accelerators from high-level programming languages such as C/C++. HLS tools can automatically generate hardware description language (HDL) code from high-level C/C++ code, significantly reducing the development time and effort required for hardware design. However, HLS tools typically support only a subset of the C/C++ language features and constructs, and certain programming patterns may not be synthesizable. The performance bottlenecks identified in the previous step can be expected to be highly optimized for the target general-purpose computing platform, and thus, may use complex programming constructs and patterns that are not compatible with HLS tools like multi-dimensional pointers, dynamic memory, STL containers etc. Therefore, the identified bottlenecks must be refactored to ensure compatibility with HLS tools while preserving their functionality.

A3D addresses this challenge with a three step process. The first step involves the *Snapshot Engineer* agent that utilizes the data structure dependencies identified by the Bottleneck Analyst to take snapshots of the input and output datastructures of the bottleneck functions during workload execution. The second step involves the *Harness Developer* agent that implements an isolated test harness for the bottleneck function and validates its correctness using the

input and output snapshots generated in the previous step. The test harness provides a controlled environment for testing and validating the bottleneck function, ensuring that it behaves as expected when refactored and optimized for HLS. The snapshot engineer and harness developer agents are designed to work in collaboration, with the snapshot engineer providing the necessary data structure coverage so that the harness developer can implement an effective test harness. If any discrepancies are found during validation, the agents iteratively refine data capture and test harness implementation until numerical correctness is achieved.

The third step involves the *HLS Preparer* agent that refactors the bottleneck function to ensure compatibility with HLS tools, by replacing unsupported programming constructs. Moreover, HLS tools provide their own libraries for optimized implementations of floating and fixed-point number formats, mathematical functions, and data structures. The HLS Preparer agent uses its agentic RAG capabilities (§3.4) to understand the HLS-optimized counterparts of the number formats and math functions used in the bottleneck function. These necessary refactorings are quite complex and require

deep understanding of both the algorithmic needs of the application and HLS tool capabilities. Therefore, A3D breaks down the HLS preparation task into smaller manageable sub-tasks, each handled by dedicated specialist agents. The HLS Preparer agent orchestrates this process by collaborating with the specialist agents. For example, Figure 2 illustrates how an HLS Preparer agent refactors standard floating point data types to HLS-compatible data types by performing a natural language query to identify the appropriate replacements. To further streamline this process, AST-based tools (§3.4) automate common refactorings such as pointer to array conversions and literal typecastings. Specialist agents leverage these tools to perform the required refactorings in a systematic manner.

At the end of the preparation phase, A3D produces an HLS-compatible version of the bottleneck function, with statically defined data structures and HLS-compatible number formats and math functions, along with a validated test harness for the function.

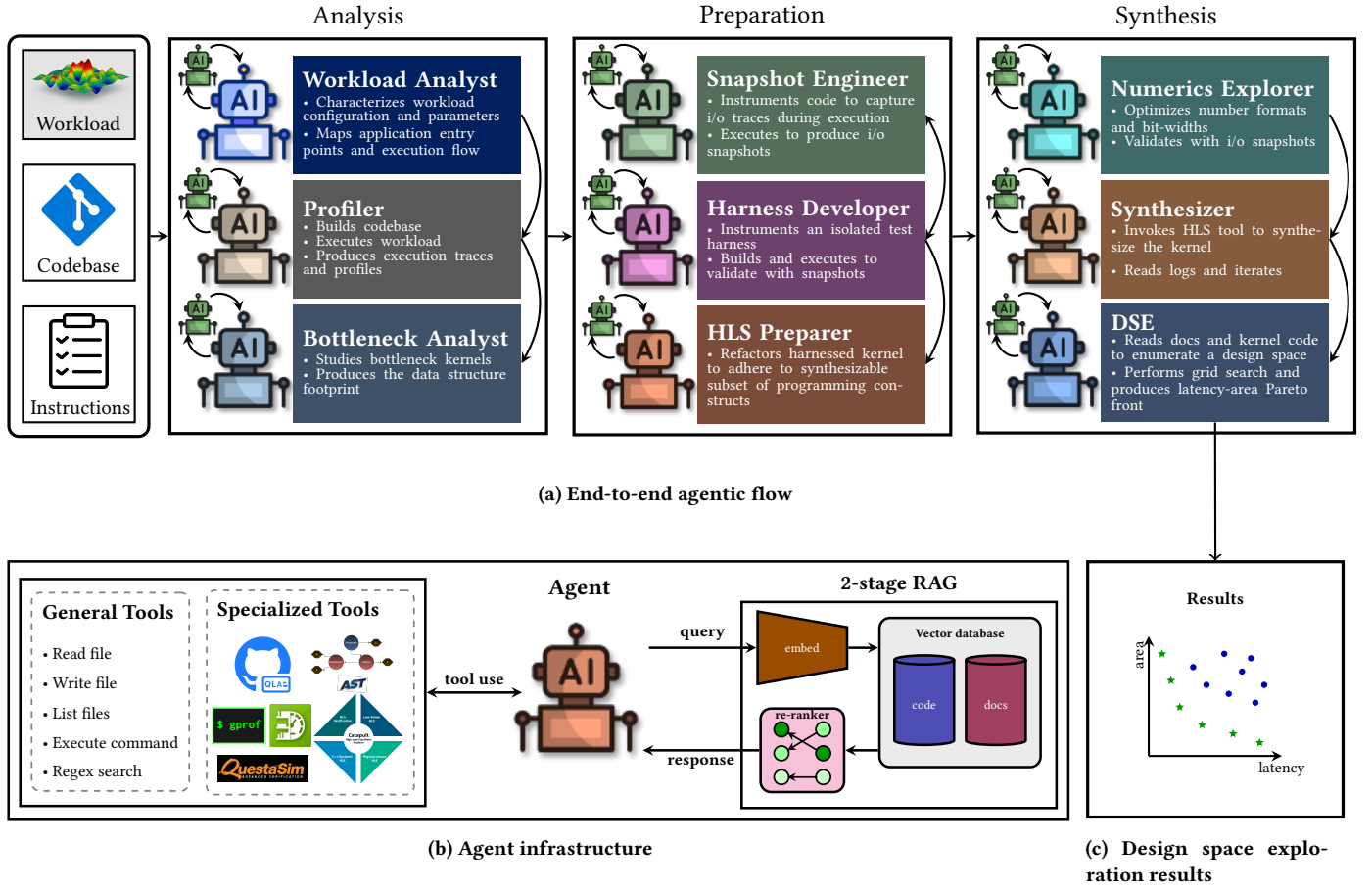


Figure 1: A3D: Agentic AI flow for Accelerator Design. (a) End-to-end multi-agent pipeline spanning three phases: Analysis, Preparation, and Synthesis (b) Shared agent infrastructure including general-purpose and specialized tools alongside a two-stage RAG pipeline over the codebase and EDA tool documentation. (c) Latency–area design space produced by the DSE agent providing a diverse set of Pareto-optimal accelerator designs.

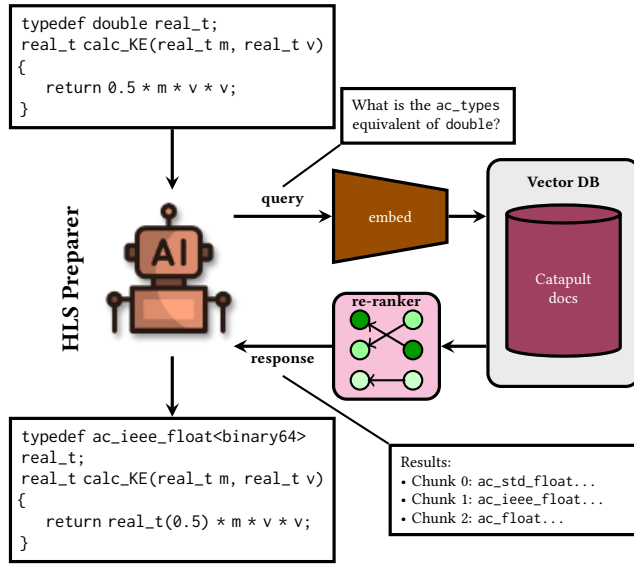


Figure 2: HLS Preparer agent refactoring floating-point types to HLS-compatible types via agentic RAG. The agent queries the Catapult documentation vector database for the `ac_types` equivalent of `double` and rewrites the source code with the retrieved replacement.

3.3 Synthesis Phase

One of the key advantages of building custom hardware accelerators is the ability to choose number formats and bitwidths that are tailored to the application’s requirements and to build the necessary data paths and compute units accordingly. This flexibility allows for significant improvements in performance, area, and energy efficiency compared to general-purpose computing platforms that use fixed number formats and bitwidths. Thus, as part of the HLS phase, the *Numerics Explorer* agent explores different number formats and bitwidths for each data element in the bottleneck function with the goal of minimizing bitwidths while preserving the functional correctness of the application. While the numerics explorer takes a trial-and-error approach in this work by iteratively reducing the bitwidths, it should be feasible to create tools based on formal methods that the agent can leverage to perform this task more systematically.

Once the number formats and bitwidths have been determined, the *Synthesizer* agent performs high-level synthesis of the prepared bottleneck function using the Catapult HLS tool. In case of synthesis failures, the synthesizer agent analyzes the synthesis reports and error logs, identifies the root causes of the failures, and finds appropriate solutions using its agentic RAG capabilities to explore the HLS tool documentation. The synthesizer agent iteratively refines the HLS-compatible code to achieve successful synthesis.

HLS tools provide several optimization directives, such as loop unrolling, pipelining, and array partitioning, that can be used to explore the speed-area tradeoff space of potential hardware accelerator designs. The *Design Space Enumerator* agent studies the bottleneck code and the optimization directives supported by the HLS tool to enumerate a diverse set of optimization directives and

their configurations that can be applied to the bottleneck function during synthesis.

The DSE agent encodes its analysis into a structured YAML specification that declaratively describes the optimization dimensions (e.g., design goal, clock period, loop unrolling factors) and their option values, along with any implied constraints such as memory interleaving requirements. A3D provides DSE generation tools that parse this YAML specification, perform combinatorial expansion across all dimensions, and emit one HLS directive (Tcl) file per design point along with a summary manifest. This deterministic tool chain ensures that every generated configuration is syntactically correct, a task that would be highly error-prone if performed generatively by the LLM across hundreds of configurations.

A grid search is then performed by synthesizing every design point, reporting the performance and area metrics for each, and extracting the Pareto-optimal set of designs. A3D provides a DSE execution tool that manages the parallel synthesis of all design points, handling job scheduling across local and remote compute nodes, Catapult invocation with the appropriate directive file for each design point, and log collection. The grid search is embarrassingly parallel and provides complete coverage of the feasibility boundary between viable and infeasible designs. A3D currently employs this grid search strategy for design space exploration, but more advanced techniques such as Bayesian optimization [26] or evolutionary algorithms [25] can be integrated into the DSE execution tool to improve the efficiency of design space exploration.

3.4 Agent Infrastructure

The specialist agents described above are supported by a shared infrastructure layer comprising custom tools, an agentic RAG pipeline, adversarial verification loops, and a top-level orchestrator. This infrastructure is critical to achieving reliable end-to-end automation despite the inherent limitations of LLMs.

3.4.1 Custom and Pre-existing Tools. The agents leverage general-purpose tools for file system exploration, reading and writing to files, and running shell commands. A3D also invokes pre-existing tools such as `gprof` for profiling and the Catapult HLS tool for high-level synthesis. Additionally, A3D implements task-specific custom tools (Table 2) for code analysis using CodeQL, code refactoring using Abstract Syntax Tree (AST) transformations and helper tools for performing design space exploration. These tools provide deterministic, repeatable transformations that complement the generative capabilities of the LLM agents, eliminating a significant source of errors in mechanical code transformations.

3.4.2 Agentic RAG Pipeline. A3D employs a two-stage retrieval pipeline to enable agentic RAG capabilities for the agents to semantically explore the application codebase and the proprietary EDA tool documentation. For example, when an agent encounters an unfamiliar construct or needs to identify the HLS-compatible counterpart of a standard library function, it issues a natural language query that is converted into an embedding, matched against vector databases populated with the codebase and tool documentation, and re-ranked for relevance. The top results are provided to

Table 2: Custom tools implemented for A3D. These tools provide deterministic, repeatable transformations that complement the generative capabilities of the LLM agents.

Tool	Phase	Function
codeQLAnalysis	Analysis	Static analysis of codebase to extract data-structure and function dependencies of bottleneck kernels
staticMem	Preparation	AST-based conversion of dynamic memory (multidimensional pointers, new[]) to statically-sized C-style arrays with compile-time constants
literal-Typecast	Preparation	AST-based insertion of explicit type casts on all numeric literals to prevent implicit conversion ambiguities
genDSE	Synthesis	Parses YAML design space specification, performs combinatorial expansion across all dimensions, emits per-design-point Tcl directive files
runDSE	Synthesis	Parallel synthesis execution across local and remote compute nodes with job scheduling, failover, and log collection

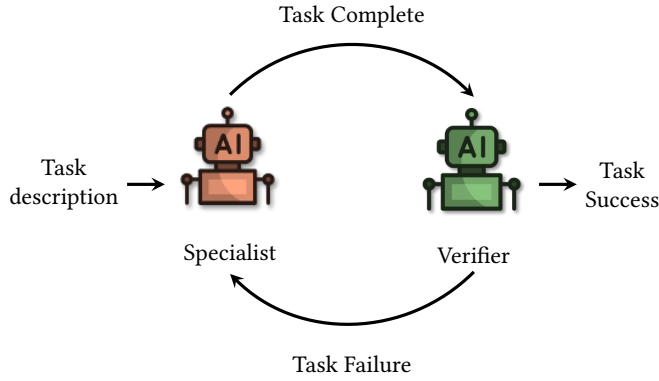


Figure 3: Specialist–Verifier iterative process loop. The verifier adversarially evaluates the specialist’s deliverables and returns critical feedback on errors or shortcomings, looping until quality standards are met.

the agent as context for the assigned task. This pipeline is essential for bridging the gap between the LLM’s training data and the proprietary knowledge required for hardware design.

3.4.3 Specialist–Verifier Loop. Given the complexity of the tasks involved in accelerator design and the stochastic nature of LLMs, it is inevitable that the outputs produced by the specialist agents may contain errors or may not meet the desired quality standards or may not meet the expected list of deliverables. To add robustness to the A3D flow and eliminate the need for human intervention, A3D employs an iterative process loop with specialist and verifier agents. Verifier agents are designed to adversarially evaluate the outputs produced by specialist agents, identify any errors, shortcomings or false claims, and provide critical feedback to the specialist agents

for task resumption and refinement. Figure 3 illustrates this process loop. The transitions between the specialist and verifier agents are automatic and the loop continues until the verifier agent validates that the deliverables produced by the specialist agent meet the desired outcomes. While this process loop can potentially lead to longer task completion times due to multiple iterations, we empirically find that it significantly improves the quality of the outputs and significantly reduces the need for human intervention.

3.4.4 Orchestration. To orchestrate this end-to-end flow with full autonomy, A3D employs the *Interfacer* agent, the top-level orchestrator. The Interfacer sequences the three phases, passes accumulated context (analysis reports, extracted artifacts, validation results) from one phase to the next, and re-invokes specialist agents as required. Crucially, the Interfacer operates at the phase level; within each phase, individual agents may themselves coordinate sub-tasks (as described above for HLS preparation), but the Interfacer is responsible for the overall flow progression and inter-phase handoff.

4 Results

In this section, we present the results of evaluating A3D. First, we characterize the benchmarks and the complexity of the kernels identified by A3D from real-world scientific applications (§4.1). We then present the end-to-end pipeline results (§4.2) demonstrating that A3D autonomously transforms complex scientific codebases into synthesized hardware accelerator designs, and compare A3D-generated accelerator designs against an edge GPU (§4.3). Next, we present a rigorous ablation study that quantifies the contribution of each architectural layer in A3D’s HLS preparation phase (§4.4), and finally walk through the end-to-end agentic flow qualitatively (§4.5).

4.1 Benchmarks and Kernel Characteristics

We evaluate A3D on two flagship scientific computing applications: LAMMPS [28], a molecular dynamics (MD) simulation toolkit, and QMCPACK [13], a quantum Monte Carlo (QMC) application. LAMMPS simulates systems of atoms that interact with each other through force functions called *interatomic potentials*: at each timestep the simulator evaluates forces on every atom and updates their positions. The force evaluation is the most computationally expensive step and the primary target for hardware acceleration. QMCPACK simulates the behavior of electrons in materials by stochastically sampling a mathematical function (the *wavefunction*) that describes the quantum state of the system; the computational cost is dominated by evaluating particle–particle interaction energies across the simulated system. Both applications are large-scale C++ codebases with hundreds of source files, deep class hierarchies, and pervasive use of dynamic memory, templates, and pointer-based data structures. These are representative of the software complexity that A3D is designed to handle. We use representative workloads for both applications, scaled down to run on a single CPU core for the purpose of demonstrating the full workflow.

4.1.1 LAMMPS Kernels. Five LAMMPS workloads were selected to span a progression of interatomic potential complexity: *lj*, *eam*, *rhodo*, *snap*, and *reaxff*. The simplest (*lj*) computes forces based

only on the distance between pairs of atoms; the most complex (*reaxff*) models the formation and breaking of chemical bonds, requiring interactions that simultaneously involve four atoms. The performance-critical kernels identified by A3D span two fundamental tasks: force evaluation and neighbor-list construction.

`PairLJCut::Compute()` evaluates the Lennard-Jones pair potential, the simplest widely-used force function, where the force between two atoms depends only on their distance. A cutoff distance is imposed so that only nearby pairs interact, which avoids the $O(N^2)$ cost of evaluating all pairs but requires a *neighbor list* data structure that records, for each atom, which other atoms are close enough to interact. The kernel iterates over each atom’s neighbor list, an irregular, input-dependent data structure, computing pairwise distances, forces, and energies. A physics symmetry (equal and opposite forces between each pair) halves the computation but introduces concurrent read-modify-write updates to force arrays, and the distance-based cutoff leads to irregular control flow. This kernel dominates the execution time in the *lj* workload.

`NPairBin::Build()` constructs the neighbor list itself, a spatial indexing structure shared by all force-evaluation kernels. The algorithm partitions the simulation domain into spatial bins and, for each atom, tests candidates from adjacent bins against a distance threshold. This kernel is characterized by multi-level pointer indirection (`int**` neighbor arrays with runtime-determined dimensions), dynamic output sizes (each atom has a different number of neighbors depending on local conditions), and indirect memory addressing through bin-to-atom mappings. It is the dominant bottleneck in the *eam* workload (which simulates metallic systems where force computation is less expensive relative to neighbor-list construction) and the *rhodo* workload (which simulates a biological protein).

`SNA::compute_uarray()` computes mathematical descriptors of each atom’s local neighborhood for a machine-learned force function (SNAP). Instead of using a fixed formula, SNAP uses descriptors that are projections of the neighbor arrangement onto a set of basis functions as inputs to a learned model that predicts forces. The descriptor computation involves deeply nested loops over basis function indices, with complex-valued arithmetic and transcendental functions (`sin`, `cos`) in the inner loop. A configuration parameter (`twojmax=8`) directly controls the loop depth and, consequently, the computational intensity. While compact in source lines, the mathematical density and loop nesting make this kernel a challenging target for HLS.

`ReaxFF::Torsion_Angles()` evaluates the energy contribution from groups of four bonded atoms in a *reactive* force field that can model the formation and breaking of chemical bonds. This capability requires tracking the bonding state of every atom and evaluating multi-body interaction terms (two-body, three-body, and four-body). The torsion angle term computes the energy penalty for deviations from preferred dihedral angles across four bonded atoms (*i-j-k-l*). Because bonds can form and break during the simulation, the set of valid four-atom groups is determined dynamically from the evolving bond topology, producing a four-deep nested loop hierarchy: the outermost loop iterates over all 2,048 atoms, the second over each atom’s bond partners (~20 per atom), the third over angle partners (~10), and the innermost over torsion partners

(~10). The innermost loop body contains ~200 floating-point operations including transcendental functions. The kernel exhibits $K=4$ windowed memory access patterns on the atom position array (simultaneously accessing atoms *i*, *j*, *k*, and *l*), accumulator dependencies on force and energy arrays, and indirect indexing through bond and interaction lists with runtime-determined bounds. This combination of deep nesting, irregular memory access, and accumulation patterns makes it among the most challenging kernels for HLS synthesis considered to-date.

4.1.2 QMCPACK Kernels. QMCPACK uses a stochastic algorithm to compute the energy of electrons in a material. A key computational challenge is evaluating the electrostatic interaction between charged particles: because this interaction extends to infinite range (unlike the cutoff-based forces in LAMMPS), it cannot be computed by simply summing over nearby pairs. Instead, the interaction is calculated through Ewald summation, which splits it into a short-range part (handled in real space) and a long-range part (handled in frequency space via Fourier transforms). The two kernels identified by A3D both arise from the frequency-space (long-range) part of this computation. Rather than targeting the dense linear algebra (BLAS) operations that dominate many quantum chemistry codes, we prioritize these non-BLAS-like kernels with irregular computation patterns to better demonstrate the breadth of A3D’s capabilities.

`LRHandlerBase::Evaluate()` computes the long-range energy contribution. The Fourier-space coefficients are pre-computed; the kernel itself is a pure accumulation loop that computes weighted dot products of pre-computed arrays grouped into shells of varying size. For each shell, the inner loop accumulates element-wise products across a variable number of elements, and the outer loop multiplies each shell’s sum by a pre-computed weight.

`StructFact::computeRhok()` computes the frequency-space representation of the particle distribution, which serves as the input to the energy evaluation above. For each particle, the kernel computes the dot product of the particle’s position with each frequency-space vector, evaluates cosine - sine pairs of the result, and accumulates the complex exponential into per-species output arrays. The kernel processes frequency-space vectors in blocks of 512, with the block count and final block size determined at runtime. The HLS challenges include CORDIC-based transcendental functions, species-based indirect write targeting (a particle’s species ID determines which output array receives the accumulation), and data-dependent loop bounds with early-exit semantics for the blocked processing.

4.1.3 Comparison with Prior Work. Beyond the workflow coverage gap discussed in §2 (Table 1), the kernels themselves present qualitatively different challenges from the benchmarks used by prior LLM-driven HLS frameworks. C2HLSC [5], SAGE-HLS [14], HLSPilot [31], LIFT [20], and LLM-DSE [29] are evaluated on isolated benchmark kernels drawn from suites such as Polybench, MachSuite [22], and CHStone [7]. These benchmarks typically consist of compact loop nests with regular, statically determinable access patterns, fixed array dimensions, and no external data structure dependencies.

Table 3: End-to-end pipeline results for all kernels processed by A3D. Each kernel was autonomously taken through the full A3D flow, spanning workload analysis, bottleneck identification, code refactoring, HLS synthesis, and design space exploration, with zero human intervention.

Kernel	App	DSE points	Successful syntheses	Synthesis Success Rate	Pareto-optimal designs
PairLJCut::Compute()	LAMMPS	1080	61	5.6%	8
NPairBin::Build()	LAMMPS	240	136	56.7%	15
SNA::compute_uarray()	LAMMPS	180	61	33.9%	11
ReaxFF::Torsion_Angles()	LAMMPS	648	109	16.8%	7
PairEAM::Compute()	LAMMPS	504	72	14.3%	7
PairLJCut::k_lj_fast()	LAMMPS	486	295	60.7%	16
StructFact::computeRhok()	QMCPACK	384	289	75.3%	15
LRHandlerBase::Evaluate()	QMCPACK	54	49	90.7%	13

In contrast, the kernels processed by A3D exhibit several dimensions of complexity absent from these benchmark suites. *Data-structure depth*: functions operate on class member variables accessed through multi-level pointer indirection (up to `****fbp` in `ReaxFF`), arrays of nested structs containing struct members (e.g., `bond_data` embedding a `bond_order_data` struct with 20 fields), and complex data dependencies. *Runtime-determined sizing*: array dimensions are not compile-time constants but are derived from simulation parameters, requiring snapshot-based size discovery and static over-provisioning (e.g., 168 atoms $\rightarrow$ 2,048-element arrays). *Incompatible programming patterns*: the kernels make heavy use of dynamic memory allocation, pointer arithmetic, STL containers, and virtual dispatch. These patterns are fundamentally incompatible with HLS tools and require systematic refactoring across dozens of constructs. *GPU-native workloads*: one kernel (`k_lj_fast`) was extracted from a CUDA implementation, requiring translation of GPU-specific idioms (thread indexing, shared memory, warp-level operations) into HLS-compatible sequential C++.

4.2 End-to-End Pipeline Results

A3D successfully processed all seven kernels identified in §4.1 through the complete pipeline with no human intervention. Table 3 presents a comprehensive summary of the end-to-end pipeline results for each kernel, spanning all three phases of the A3D flow: the analysis phase (bottleneck identification and dependency extraction), the preparation phase (snapshot capture, harness development, and HLS-compatible refactoring), and the synthesis phase (numerics optimization, HLS synthesis, and design space exploration).

4.2.1 Implementation. The agents in A3D are implemented using the Roo Code agent development toolkit [23], which provides a framework for building and executing agentic workflows. While Roo Code supports choosing the underlying LLM for each agent, we use Claude Sonnet 4.5 as the underlying LLM for all agents in A3D due to its strong reasoning and code generation capabilities. The agentic RAG pipeline is implemented using state-of-the-art embedding and re-ranking models [35], and vector databases. When the agent makes an enquiry, it is converted into a 4096-dimensional embedding using Qwen3-Embedding-8B. This embedding is then used to retrieve the top-k cosine-similar chunks from vector databases

that have been populated with the codebase and the EDA tool documentation, respectively. The retrieved chunks are then re-ranked using Qwen3-Reranker-8B to reorder them based on their relevance to the original query. The top-m re-ranked chunks are then provided to the agent as context for the assigned task. The HLS tool used in A3D is the Catapult HLS tool from Siemens, a widely used commercial HLS tool that supports C++ input and generates RTL implementations.

4.2.2 Design Space Exploration. A3D automatically identifies optimization opportunities for each kernel, such as loop unrolling, loop pipelining, and array partitioning factors, alongside global synthesis settings like target clock period and optimization goals (speed vs. area), and enumerates a combinatorial design space from these dimensions. Each design point is synthesized independently and a Pareto-optimal set of designs is extracted. Across all eight kernels, A3D enumerated a total of 3,216 design points and successfully synthesized 1,036 of them, yielding 92 Pareto-optimal designs (Table 3).

Synthesis success rates vary widely across kernels, ranging from 5.6% to 90.7%. The variation is driven primarily by memory interface complexity: kernels with many distinct arrays requiring simultaneous access (e.g., `PairLJCut` accesses over a dozen arrays including position, force, neighbor lists, and coefficient tables, each requiring independent memory ports under unrolling or pipelining) have a larger fraction of infeasible configurations, while kernels with fewer arrays and more regular access patterns achieve higher success rates. The two QMCPACK kernels consistently achieve higher synthesis rates (75.3% and 90.7%) than the LAMMPS kernels, reflecting their simpler data structure dependencies and more regular memory access patterns.

4.3 Accelerator vs. Edge GPU Comparison

To demonstrate the practical value of A3D-generated accelerators, we compare them against an edge GPU. A3D processed the GPU-accelerated LAMMPS workload, identified the `k_lj_fast()` CUDA kernel, which accelerates the Lennard-Jones computation with GPUs, and generated accelerator designs through its full pipeline. A3D successfully re-implemented the CUDA kernel into an HLS-compatible C++ kernel, thus enabling the direct integration of the generated accelerator into the original GPU-accelerated execution

Table 4: Comparison of A3D-generated accelerator designs against the NVIDIA Jetson AGX Orin for the PairLJCut kernel. A3D power and EDP are scaled from 22 nm to 8 nm (0.30× power factor) for an iso-technology comparison.

Platform	Latency	Power	EDP
Jetson AGX Orin	2.1 ms	3.6 W	15.9 μ Js
A3D Design (low latency)	7.85 ms	1.04 W	64.1 μ Js
A3D Design (low power)	618 ms	0.87 W	0.33 Js

flow. This demonstrates A3D’s ability to accelerate workloads that are already accelerated on GPUs, which is a common scenario in scientific computing where researchers often use GPU-accelerated libraries but still face bottlenecks, as the irregular compute patterns of these kernels make them a poor fit for GPU acceleration.

The generated accelerators were synthesized using the 22 nm technology from Global Foundries. An data-driven architecture simulator was implemented to estimate latency and energy consumption of each design point. Since the Jetson AGX Orin uses Samsung’s 8 nm process, we scale the A3D power figures to 8 nm using a conservative 0.30× factor.

The results are summarized in Table 4. The A3D low-latency design achieves a power consumption of 1.04 W—a 3.5× power advantage over the Jetson’s 3.6 W—with a comparable energy per execution (8.2 mJ vs. 7.6 mJ) and an EDP of 64.1 μ Js compared to the Jetson’s 15.9 μ Js. The remaining latency and EDP gap is driven by the Jetson’s higher degree of parallelism. The fastest A3D design occupies only 49,494 μ m², which is a small fraction of the area of the Orin SoC. Crucially, these accelerator designs were generated with *zero human intervention* from a CUDA kernel, which is a novel capability.

4.4 Ablation Study: Impact of Agentic Scaffolding

A central claim of A3D is that its multi-layered agentic architecture, comprising task decomposition into specialist agents, augmentation with custom tools, and adversarial verification loops, is essential for reliable agentic hardware generation. To rigorously validate this claim, we conduct an ablation study on the PairLJCut::Compute() kernel from LAMMPS, measuring the success rate of the HLS preparation phase under four progressively richer agent design configurations.

4.4.1 Experimental Design. We define four ablation configurations, each adding one architectural layer atop the previous:

- **Config A — Monolithic Agent:** A single monolithic HLS Preparer agent performs all preparation tasks without task decomposition.
- **Config B — Multi-Agent:** The preparation task is decomposed into six sequential phases, each handled by a dedicated specialist agent coordinated by an orchestrator: (0) datastructure pruning, (1) memory interface refactoring, (2) type conversion to `ac_types`, (3) loop labeling for HLS directives, (4) math function replacement with `ac_math` equivalents, and (5) construct verification and cleanup.

Table 5: Bayesian posterior analysis of success rates across ablation configurations. n : completed trials, k : passed trials, CI: 95% credible interval.

Config	n	k	Pass Rate	Post. Mean	95% CI	$P(\theta > 90)$
(A) Monolithic	75	24	32.0	32.5	[22.5, 43.3]	0.0
(B) Multi-Agent	73	42	57.5	57.3	[46.1, 68.2]	0.0
(C) + Tools	60	45	75.0	74.2	[62.7, 84.2]	0.0
(D) + Verifiers	30	29	96.7	93.8	[83.3, 99.2]	0.83

- **Config C — Agents + AST Tools:** Along with the multi-agent decomposition, we integrate AST-based tools for deterministic code transformations to augment the agents’ capabilities. Agent 1 invokes the `staticMem` tool for kernel interface refactoring and Agent 2 invokes the `literalTypecast` tool for systematic typecasting of numeric literals.
- **Config D — Agents + Tools + Verifiers:** The full A3D architecture comprising of decomposed specialist agents augmented with AST tools along with an adversarial verification loop for each specialist phase. After each specialist agent completes its task, a dedicated verifier agent performs targeted checks to identify any errors or omissions in the expected deliverables of that phase, providing feedback for iterative refinement until the specialist’s output passes all verification checks.

For each configuration, we execute n independent trials using identical inputs (the isolated PairLJCut::Compute() test harness implemented by Harness Developer and I/O snapshots produced by the Snapshot Engineer). Each trial is validated by a three-stage short-circuit evaluation pipeline: (1) *compile*: the refactored kernel and test harness must compile without errors, (2) *execute*: the compiled harness must run and produce outputs that match the golden outputs, and (3) *synthesize*: the HLS tool must successfully synthesize the refactored kernel. A trial passes only if all three stages succeed; evaluation halts at the first failure. We model the success rate using a Beta-Binomial Bayesian framework with a uniform Beta(1, 1) prior, employing adaptive stopping based on posterior precision ($\pm 12.5\%$) and success ($P(\theta > 90\%) > 0.9$)/futility ($P(\theta < 10\%) > 0.33$) thresholds. We use Qwen3.5-27B [21] for all agents in this ablation study to isolate the impact of architectural design choices from model capabilities. We note that Qwen3.5-27B is a state-of-the-art LLM with strong reasoning and code generation capabilities scoring 72.4% on SWE-bench-Verified [11] vs. Claude Sonnet 4.5’s 71.4%.

4.4.2 Success Rate Analysis. Figure 4 presents the posterior mean success rate for each configuration with 95% credible intervals. The results show a clear upward staircase as A3D features are added.

Table 5 presents the full Bayesian analysis. The posterior mean success rate increases from 32.5% (Config A) to 93.8% (Config D), with each architectural layer contributing a substantial improvement. Notably, only the full A3D architecture (Config D) achieves a nonzero probability of exceeding a 90% success rate ($P(\theta > 90\%) = 0.83$), while all other configurations have effectively zero probability of reaching this threshold.

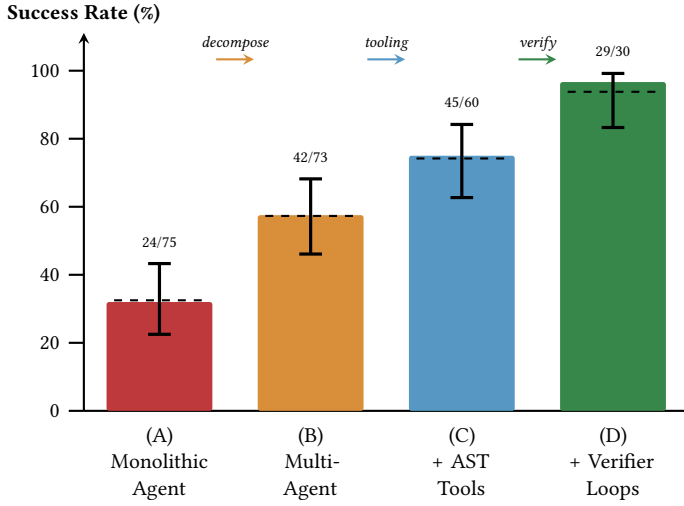


Figure 4: Success rates for the HLS preparation phase under four ablation configurations. Bar heights indicate the raw pass rate (k/n); dashed lines mark the Bayesian posterior mean; error bars show 95% credible intervals. Each layer of A3D’s architecture—task decomposition (A→B), tool augmentation (B→C), and verification loops (C→D)—contributes a measurable improvement in reliability.

Going from Config A to Config B demonstrates the value of **task decomposition**: the pass rate nearly doubles from 32.0% to 57.5%. A monolithic agent cannot manage the cognitive complexity of refactoring 35 unsupported constructs across nearly 1500 AST nodes within a single context window, frequently triggering context condensation and losing track of earlier transformations or introducing inconsistencies. The improvement from Config B to Config C validates **tool augmentation**: the pass rate rises from 57.5% to 75.0%. While LLMs can attempt code transformations like literal typecasting and pointer-to-array conversion generatively, the AST-based tools handle these transformations deterministically and reliably, eliminating a significant source of errors. Finally, the gain from Config C to Config D validates the **specialist–verifier loop**: the pass rate jumps from 75.0% to 96.7%. Adversarial verification catches subtle errors that specialist agents miss, such as incomplete transformations. This final layer ultimately pushes A3D into the high-reliability regime, as reflected by the $P(\theta > 90\%) = 0.83$ in Config D versus zero for all other configurations.

4.4.3 Cost-Efficiency Analysis. While simpler configurations are cheaper, they are less reliable. Table 6 presents the mean cost and efficiency metrics for each configuration. Mean per-trial cost increases monotonically from \$2.65 (Config A) to \$7.33 (Config D), driven primarily by higher token consumption. Config D processes 23M input tokens and 171K output tokens per trial, roughly 2.5× and 3× that of Config A, respectively. The higher token counts reflect the overhead of multi-agent coordination, tool invocations, and verifier feedback loops.

The *cost per successful outcome* (mean cost divided by pass rate) is minimized by Config C at \$5.43, owing to its arsenal of tools. Config D incurs a higher cost per success (\$7.58) due to verifier loop overhead. However, reliability is paramount for end-to-end automation, and Config D is the only configuration for which $P(\theta > 90\%) > 0$, indicating that the verifier loop is necessary to achieve high-confidence automation. At a 75% pass rate (Config C), one in four runs requires human intervention; at 97% (Config D), this drops to approximately one in thirty-three, a reduction that is critical for unmanned deployment.

We report that the observations from this ablation study guided the design of A3D’s architecture based on all three principles of task decomposition, tool augmentation, and verification loops to achieve similar levels of reliability at every step of the A3D workflow.

4.4.4 Verifier Loop Behavior. To understand *where* the verifier loop’s overhead materializes, we parse the agent execution traces from Config D into *specialist episodes*, defined as the subsequence of mode transitions between when the orchestrator dispatches to a specialist and when control returns to the orchestrator. For each of the six preparation phases, we measure the number of verifier rounds (specialist→verifier transitions) per episode and the *single-pass rate*: the fraction of episodes in which the specialist’s output was accepted by the verifier on the first attempt.

Table 7 presents the per-phase results across all 30 completed Config D trials. Phases 0 (datastructure pruning), 3 (loop labeling), and 5 (construct verification) achieve a 100% single-pass rate: the verifier never rejects the specialist’s output. Phase 4 (math function replacement) is nearly as stable at 96.7%, with only one trial requiring a second verifier round. In contrast, Phases 1 (memory interface refactoring) and 2 (type conversion) both achieve a 76.7% single-pass rate, with Phase 2 exhibiting the highest maximum verifier rounds (3) and transitions (6) of any phase. These phases involve the most complex, multi-site code transformations: converting multi-level pointer indirection to static arrays (Phase 1) and replacing all numeric types with HLS-compatible `ac_types` (Phase 2).

Table 6: Cost and efficiency metrics across ablation configurations. Cost per success is computed as mean cost / pass rate.

Config	n	k	Mean Cost (\$)	Total Cost (\$)	Mean Time (min)	Mean Tokens In	Mean Tokens Out	Mean API Calls	Cost per Success (\$)
(A) Monolithic	75	24	2.65	198.75	12m13s	9,311,098	58,166	86	9.17
(B) Multi-Agent	73	42	3.66	267.18	20m0s	11,479,576	91,715	169	6.37
(C) + Tools	60	45	4.07	244.20	23m20s	12,662,270	112,522	191	5.43
(D) + Verifiers	30	29	7.33	219.90	37m19s	23,071,153	170,735	303	7.58

Table 7: Per-phase verifier loop behavior in Config D (30 trials). Single-pass rate: fraction of episodes where the verifier accepted the specialist’s output on the first attempt.

Phase	Mean Verifier Rounds	Max Verifier Rounds	Single-Pass Rate
0 – Pruning	1.00	1	100%
1 – Memory	1.23	2	76.7%
2 – Types	1.27	3	76.7%
3 – Loop labels	1.00	1	100%
4 – Math	1.03	2	96.7%
5 – Verification	1.00	1	100%

Importantly, the per-phase fragility profile is *kernel-specific*. The `PairLJCut::Compute()` kernel used in this ablation has relatively simple data structures and straightforward arithmetic. In contrast, the `ReaxFF::Torsion_Angles()` kernel described in §4.5 requires processing far more complex data structures and transcendental math functions making the verifier crucial for phases 0 and 4 as well. These results should therefore be interpreted as characterizing the agentic flow on one representative kernel rather than as universal difficulty rankings across phases.

4.5 End-to-End Flow: A Case Study

To illustrate how A3D operates in practice, we present its end-to-end processing of the `ReaxFF::Torsion_Angles()` kernel from LAMMPS. This kernel evaluates torsion angle energy contributions in the Reactive Force Field and is representative of the most challenging kernel in our benchmark suite.

4.5.1 Analysis Phase. The *Workload Analyst* began by exploring the LAMMPS codebase using semantic search. It identified the *reaxff* workload as exercising the reactive force field potential, mapping the execution flow from the main simulation loop through the `ReaxFF` force computation routines to the torsion angle evaluation kernel.

The *Performance Profiler* compiled LAMMPS with profiling instrumentation and executed the *reaxff* workload. The profiling results identified `ReaxFF::Torsion_Angles()` as a dominant performance bottleneck, owing to its deeply nested loop structure.

The *Bottleneck Analyst* then used CodeQL to perform static analysis, revealing that `Torsion_Angles()` depends on multiple complex data structures spanning several class hierarchies. The Bottleneck Analyst dynamically generates CodeQL queries tailored to each kernel it analyzes. Figure 5 shows the query it formulated for `Torsion_Angles()`. The query identifies every variable accessed by the kernel, both directly and through functions it calls (one level of call depth), and classifies each access as INPUT or OUTPUT. This produces a complete I/O footprint: variable name, type, and data-flow direction. For `Torsion_Angles()`, the query identified 25 array parameters spanning 7 struct types across 4 class hierarchy levels, producing the dependency map that guides the Snapshot Engineer (which variables to capture) and the Harness Developer (which data structures the isolated test harness must replicate). Figure 6 shows a subset of these data structures as defined in the

```
import cpp

from Function f, Variable v,
    VariableAccess va, string usage,
    Function calledFunc
where
    f.getName() = "Torsion_Angles" and
    f.getFile().getBaseName() =
        "reaxff_torsion_angles.cpp" and
    (
        (va.getEnclosingFunction() = f) or
        (exists(FunctionCall fc |
            fc.getEnclosingFunction() = f and
            fc.getTarget() = calledFunc and
            va.getEnclosingFunction() = calledFunc
        ))
    ) and
    va.getTarget() = v and
    if va.isUsedAsLValue()
    then usage = "OUTPUT"
    else usage = "INPUT"
select va,
    v.getName() + ", " +
    v.getType().toString() + ", " + usage
```

Figure 5: CodeQL query dynamically generated by the Bottleneck Analyst to extract the I/O data-structure footprint of `Torsion_Angles()`. The query traverses one level of call depth to capture variables accessed by helper functions.

original LAMMPS source (`reaxff_types.h`). The Bottleneck Analyst then performs dynamic analysis to determine the runtime sizes and behavior of these data structures with the representative workload.

4.5.2 Preparation Phase. The *Snapshot Engineer* instrumented code at the kernel call site to capture the complete input and output state of `Torsion_Angles()` at the function boundary during workload execution. The captured input snapshot (8.9 MB) includes atom positions, bond and three-body interaction lists (40,320 bonds, 68,659 three-body interactions), workspace arrays, and force field parameters; the output snapshot (951 KB) captures all modified state.

The *Harness Developer* created an isolated test harness comprising three files (`bottleneck.cpp`, `bottleneck.h`, `main.cpp`) that reads the captured snapshots, invokes the extracted kernel, and validates outputs against the golden reference from the output snapshot. The harness was iteratively refined in collaboration with the Snapshot Engineer until numerical equivalence was achieved. The *HLS Preparer* then performed the most challenging step: refactoring the kernel’s unsupported constructs, such as multi-level pointer indirection, dynamic memory allocation, standard library math functions and floating point numbers, with HLS-compatible alternatives. This was orchestrated across six specialist phases, each followed by adversarial verification:

- (0) **Datastructure Pruning:** Removed 28 unused fields across 7 structs and converted a union to a struct, thereby simplifying the kernel’s type system.

```

// Primitive types
using rvec = double[3];
using ivec = int[3];

// Atom data
struct reax_atom {
    rc_tagint orig_id;
    int type; char name[8];
    rvec x, v, f; double q;
    int Hindex, num_bonds,
        num_hbonds;
};

// Force field parameters
struct global_parameters {
    int n_global, vdw_type;
    double *l;
};
struct four_body_parameters {
    double V1, V2, V3;
    double p_tor1, p_cot1;
};
struct four_body_header {
    int cnt;
    four_body_parameters
        prm[REAX_MAX_4BODY_PARAM];
};
struct reax_interaction {
    int num_atom_types;
    global_parameters gp;
    single_body_parameters *sbp;
    two_body_parameters **tbp;
    three_body_header ***tbp;
    hbond_parameters ****hbp;
    four_body_header ****fbp;
};

// 1st arg: system
struct reax_system {
    reax_interaction reax_param;
    int n, N, numH;
    int local_cap, total_cap, Hcap;
    reax_atom *my_atoms;
    int my_bonds;
    double safezone, saferzone;
    LR_lookup_table **LR;
};

// 2nd arg: control
struct control_params {
    int nthreads;
    double bond_cut;
    double nonb_cut, nonb_low;
    double hbond_cut;
    double bg_cut, bo_cut;
    double thb_cut, thb_cutsq;
    int tabulate;
    int lgflag, enobondsflag;
};

// 3rd arg: simulation data
struct energy_data {
    double e_bond, e_ov, e_un;
    double e_lp, e_ang, e_pen;
    double e_coa, e_hb;
    double e_tor, e_con;
    double e_vdw, e_ele, e_pol;
};

struct simulation_data {
    rc_bigint step;
    energy_data my_en;
};

// 4th arg: workspace storage
struct storage {
    int allocated;
    double *total_bond_order;
    double *Deltap, *Deltap_boc;
    double *Delta, *Delta_lp;
    double *Delta_lp_temp;
    double *Delta_e, *Delta_boc;
    double *Delta_val;
    double *dDelta_lp;
    double *dDelta_lp_temp;
    double *nlp, *nlp_temp;
    double *Clp, *vlpex;
    rvec *dDeltap_self;
    int *bond_mark;
    double Tap[8];
    double *CdDelta;
    rvec *f;
    rvec *forceReduction;
    double *CdDeltaReduction;
    int *valence_angle_atom_myoffset;
    double *s;
    reallocate_data realloc;
};

// 5th arg: reax_list **
struct bond_order_data {
    double B0, B0_s, B0_pi, B0_pi2;
    double Cdbo, Cdbopi, Cdbopi2;
    double C1dbo, C2dbo, C3dbo;
    double C1dbopi, C2dbopi;
    double C3dbopi, C4dbopi;
    double C1dbopi2, C2dbopi2;
    double C3dbopi2, C4dbopi2;
    rvec dB0p, dln_B0p_s;
    rvec dln_B0p_pi, dln_B0p_pi2;
    double *CdboReduction;
};

struct bond_data {
    int nbr, sym_index;
    int dbond_index;
    ivec rel_box;
    double d; rvec dvec;
    bond_order_data bo_data;
};

struct three_body_interaction_data {
    int thb, pthb;
    double theta, cos_theta;
    rvec dcos_di, dcos_dj, dcos_dk;
};

union list_type {
    three_body_interaction_data
        *three_body_list;
    bond_data *bond_list;
    far_neighbor_data *far_nbr_list;
    hbond_data *hbond_list;
};

struct reax_list {
    int allocated, n, num_intrs;
    int *index, *end_index;
    int type; list_type select;
};

```

Figure 6: A subset of the data structures involved in extern void TorsionAngles(reax_system*, control_params*, simulation_data*, storage*, reax_list), as discovered by the Bottleneck Analyst. The five function arguments expose a deep type hierarchy featuring dynamically allocated arrays, multi-level pointer indirection (four_body_header ****fbp), a union-based polymorphic list container (list_type), and nested structs.**

- (1) **Memory Interface Refactoring:** Multi-level pointer indirection in bond lists, three-body lists, and atom position arrays was converted to C-style array notation with compile-time constants, using the AST-based staticMem tool. The tool applied 105 automated transformations (90 in the kernel, 15 in the header). Array dimensions were determined from the captured snapshots and sized 10× larger to accommodate production workloads (e.g., 168 atoms → 2,048 static array, 40,320 bonds → 400,000). A subsequent verifier pass identified a residual LAMMPS: :Pair* pointer that was missed; the specialist converted it to a statically-embedded struct, achieving zero remaining pointers across all 15 data structures.
- (2) **Type Conversion:** Standard numeric types were replaced with HLS-compatible ac_types using compile guards. The agent queried the Catapult documentation RAG for appropriate type mappings (Figure 2). Nine category-specific typedefs were introduced (Energy_t, Angle_t, BondOrder_t, Distance_t, Param_t, Calc_t, AtomDeriv_t, BondDeriv_t, Real_t) to enable independent precision control in the numerics exploration phase. All numeric literals were explicitly typecast using the literalTypecast tool to prevent implicit conversion ambiguities.
- (3) **Loop Labeling:** All four nested loop structures were labeled with unique, descriptive identifiers to enable targeted application of HLS optimization directives in the synthesis phase.
- (4) **Math Function Replacement:** Five standard math functions (sin, cos, sqrt, exp, atan2) were replaced with HLS-compatible ac_math equivalents, consulting both local documentation and the Catapult RAG pipeline.
- (5) **Construct Verification:** A final verification pass ensured no remaining unsupported programming constructs, validated compilation under HLS flags, and confirmed numerical equivalence against the golden snapshots.

4.5.3 Synthesis Phase. In the synthesis phase, the numerics of data structures and computations were optimized for hardware efficiency while preserving functional correctness, before invoking Catapult HLS for RTL generation. This phase is critical for achieving a performant hardware design, as naive type choices can lead to excessive resource usage or failed synthesis.

The *Numerics Explorer* optimized number formats and bit-widths to maximize hardware efficiency while preserving functional correctness. The kernel’s nine floating-point typedefs, initially set to ac_ieee_float64 (IEEE-754 double), were converted to fixed-point representations (ac_fixed) for more efficient hardware implementation.

A key discovery during this phase was that *precision is more important than range*. The initial attempt with ac_fixed<64, 32, true> (32 integer bits, 31 fractional) produced 10.5% error in torsion energy due to insufficient fractional precision for accumulated values. Reducing the integer width to 20 bits while increasing fractional bits to 43 (ac_fixed<64, 20, true>) passed verification, since 20 integer bits (±524,288 range) suffice for all kernel values while 43 fractional bits provide the precision needed for accurate representation.

The Numerics Explorer then performed per-type bit-width optimization on each typedef independently, respecting a *scalability exception* policy: typedefs used in accumulation patterns (Energy_t, AtomDeriv_t, BondDeriv_t) were preserved at 64 bits to prevent precision loss from compounding across interactions, while optimization candidates were reduced through binary search with verification after each step:

- BondOrder_t: 64 → 17 bits (73.4% reduction) — bounded range [0, ~3]
- Angle_t: 64 → 21 bits (67.2%) — bounded range $[-\pi, \pi]$
- Param_t: 64 → 22 bits (65.6%) — constant force field parameters
- Calc_t: 64 → 45 bits (29.7%) — intermediate calculations

Two typedefs (Distance_t, Real_t) could not be optimized due to type interdependencies: they participate in vector operations (rvec_ScaledAdd) that interface with AtomDeriv_t (a scalability exception), creating a type-width coupling that the agent correctly identified and documented rather than forcing an incompatible reduction.

The *Synthesizer* invoked Catapult HLS to synthesize the prepared kernel. The baseline synthesis completed successfully in 1 hour 36 minutes, generating RTL netlists in both Verilog and VHDL, along with other artifacts such as synthesis reports, resource utilization summaries, and timing analyses.

4.5.4 Design Space Exploration. Design space exploration is broken into three phases: (0) knob identification, (1) strategy and configuration generation, and (2) execution. The goal is to systematically explore the optimization space of HLS directives (e.g., loop unrolling, pipelining, memory partitioning) to identify Pareto-optimal configurations that balance area and latency.

Phase 0 — Knob Identification. The DSE agent conducted 8 focused RAG queries to build comprehensive understanding of Catapult HLS optimization techniques, covering design goals, clock period constraints, loop unrolling, loop pipelining, and memory interleaving. It then performed manual code analysis of the kernel’s loop hierarchy and classified all loop dependencies, finding that the innermost loop exhibits weak accumulator dependencies (energy accumulations to the same location, force accumulations to different atoms), making II=1 likely achievable. Array access patterns were documented with K-values: the atom position array requires K=4 windowed reads (simultaneously accessing atoms i, j, k, l), bond lists require K=3 reads, and three-body lists require K=2 reads.

Phase 1 — Strategy and Configuration Generation. Using the extracted HLS resource paths (25 array parameters and 4 loop paths from the baseline synthesis log) and the knob identification results, the agent generated a YAML design space specification. Figure 7 shows an excerpt of this specification, illustrating its declarative structure. Each dimension defines its options with an HLS directive type, target resource path, and range of values. Memory interleaving factors were computed based on number of accesses per iteration. Multiple interleaving levels were explored for each unroll/pipeline option to map the boundary between feasible and infeasible designs. The agent then invoked the genDSE tool, which parsed this YAML

```
kernel_name: "TorsionAngles"
base_hls_tcl_file: "directives/baseline.tcl"
dimensions:
  - id: "design_goal"
    type: "DESIGN_GOAL"
    values: [area, latency]

  - id: "clock_period"
    type: "CLOCK_PERIOD"
    values: [1.0, 2.0, 3.0, 5.0, 7.0, 10.0]

  - id: "innermost_loop_unroll"
    type: "UNROLL"
    target_hls_path: ".../LOOP_TORSION_ANGLE_L"
    values: [no, 2, 4] # with interleaving variants

  - id: "innermost_loop_pipeline"
    type: "PIPELINE_II"
    target_hls_path: ".../LOOP_TORSION_ANGLE_L"
    values: [none, 1, 2] # with interleaving variants
```

Figure 7: Excerpt of the YAML design space specification generated by the DSE agent for TorsionAngles(). Each dimension declares its HLS directive type, target resource path, and the set of values to explore. The genDSE tool parses this specification and performs combinatorial expansion to emit one Tcl directive file per design point.

specification, performed combinatorial expansion across all possible combinations, and emitted 649 synthesis-ready Tcl directive files along with a CSV summary manifest.

Phase 2 — Parallel Execution. The agent invoked the runDSE execution tool, which scheduled the 649 configurations for parallel synthesis across 128 concurrent jobs distributed over local and remote compute nodes. The tool manages Catapult invocation with the appropriate directive file for each design point, handles remote job scheduling via SSH, and collects per-configuration synthesis logs. The total runtime for all 649 syntheses was 30 hours 27 minutes. Of the 649 configurations, 109 synthesized successfully

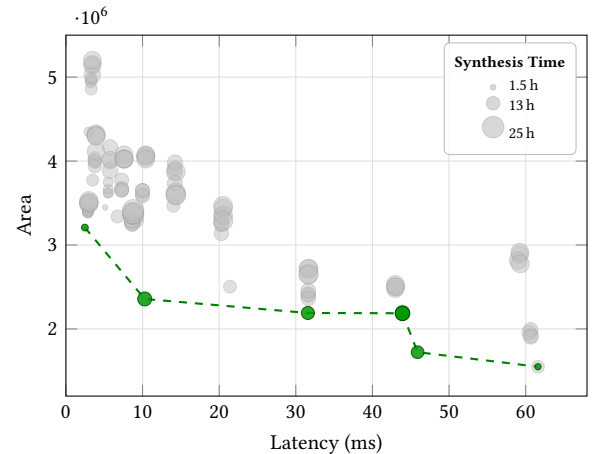


Figure 8: Design space exploration results for TorsionAngles(). Each point represents a successfully synthesized configuration; marker size encodes synthesis time (1.5–25 h). Green markers indicate the 7 Pareto-optimal designs spanning a 25× latency–2.1× area tradeoff.

(16.8% success rate). The 540 failures are expected and informative: they represent under-interleaved configurations that cannot meet scheduling constraints, validating the memory pressure model from Phase 0. Configurations with position array interleaving below $K=4$ systematically fail, confirming the windowed access analysis. Successful configurations span the full optimization space, with area-optimized designs (slow clock, lower unroll) succeeding with minimal interleaving and latency-optimized designs (fast clock, higher unroll/pipeline) requiring full interleaving. Individual synthesis times ranged from ~ 1.5 hours for simple configurations to up to 25 hours for complex designs.

Figure 8 shows the complete design space for Torsion_Angles, plotting latency against area for all 109 successfully synthesized configurations. The Pareto front identifies 7 non-dominated designs spanning a $25\times$ latency range (2.5–61.6 ms) and a $2.1\times$ area range.

5 Conclusion

We present A3D, the first agentic AI system for end-to-end automation of hardware design from application codebases to diverse accelerator architectures. Starting from a C/C++ or CUDA codebase and a representative workload, A3D autonomously profiles the application, identifies performance bottlenecks, extracts and isolates target kernels, refactors them for HLS compatibility, performs high-level synthesis, and explores the design space, all with zero human intervention. We demonstrated A3D on two production-scale scientific computing applications, processing kernels that span a wide range of HLS challenges. An ablation study validated A3D's three-layer architecture: task decomposition, deterministic tool augmentation, and adversarial specialist-verifier loops each contribute measurably to reliability, and only their combination achieves the high success rates required for unmanned deployment. These findings suggest that reliable agentic automation of complex engineering tasks requires the synergy of all three principles.

References

- [1] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2024. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*. 3119–3137.
- [2] Fengxiang Bie, Yibo Yang, Zhongzhu Zhou, Adam Ghanem, Minjia Zhang, Zhewei Yao, Xiaoxia Wu, Connor Holmes, Pareesa Golnari, David A Clifton, et al. 2024. Renaissance: A survey into ai text-to-image generation in the era of large model. *IEEE transactions on pattern analysis and machine intelligence* 47, 3 (2024), 2212–2231.
- [3] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. 2016. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits* 52, 1 (2016), 127–138.
- [4] Andrew A Chien, Allan Snively, and Mark Gahagan. 2011. 10x10: A general-purpose architectural approach to heterogeneity and energy efficiency. *Procedia Computer Science* 4 (2011), 1987–1996.
- [5] Luca Collini, Siddharth Garg, and Ramesh Karri. 2025. C2hls: Leveraging large language models to bridge the software-to-hardware design gap. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 1–24.
- [6] Yu Gao, Haoyuan Guo, Tuyen Hoang, Weilin Huang, Lu Jiang, Fangyuan Kong, Huixia Li, Jiashi Li, Liang Li, Xiaojie Li, et al. 2025. Seedance 1.0: Exploring the boundaries of video generation models. *arXiv preprint arXiv:2506.09113* (2025).
- [7] Yuko Hara, Hiroyuki Tomiyama, Shinya Honda, Hiroaki Takada, and Katsuya Ishii. 2008. Chstone: A benchmark program suite for practical c-based high-level synthesis. In *2008 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1192–1195.
- [8] John L Hennessy and David A Patterson. 2019. A new golden age for computer architecture. *Commun. ACM* 62, 2 (2019), 48–60.
- [9] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
- [10] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2026. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology* 35, 2 (2026), 1–72.
- [11] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues?. In *International Conference on Learning Representations*, Vol. 2024. 54107–54157.
- [12] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [13] Paul RC Kent, Abdulgani Annaberdiyev, Anouar Benali, M Chandler Bennett, Edgar Josué Landinez Borda, Peter Doak, Hongxia Hao, Kenneth D Jordan, Jaron T Krogel, Ilkka Kylänpää, et al. 2020. QMCPACK: Advances in the development, efficiency, and application of auxiliary field and real-space variational and diffusion quantum Monte Carlo. *The Journal of chemical physics* 152, 17 (2020).
- [14] M Khan, Nowfel Mashnoor, Mohammad Akyash, Kimia Azar, and Hadi Kamali. 2025. SAGE-HLS: Syntax-Aware AST-Guided LLM for High-Level Synthesis Code Generation. *arXiv preprint arXiv:2508.03558* (2025).
- [15] Sakari Lahti, Panu Sjövall, Jarno Vanne, and Timo D Hämäläinen. 2018. Are we there yet? A study on the state of high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 5 (2018), 898–911.
- [16] Runkai Li, Jia Xiong, Xiuyuan He, Jiaqi Lv, Jieru Zhao, and Xi Wang. 2025. ChatHLS: Towards Systematic Design Automation and Optimization for High-Level Synthesis. *arXiv preprint arXiv:2507.00642* (2025).
- [17] Grant Martin and Gary Smith. 2009. High-level synthesis: Past, present, and future. *IEEE Design & Test of Computers* 26, 4 (2009), 18–25.
- [18] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislov Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131 [cs.AI]* <https://arxiv.org/abs/2506.13131>
- [19] Ali Emre Oztas and Mahdi Jelodari. 2024. Agentic-HLS: An agentic reasoning based high-level synthesis system using large language models (AI for EDA workshop 2024). *arXiv preprint arXiv:2412.01604* (2024).
- [20] Neha Prakriya, Zijian Ding, Yizhou Sun, and Jason Cong. 2025. LIFT: Llm-based pragma insertion for HLS via GNN supervised fine-tuning. *arXiv preprint arXiv:2504.21187* (2025).
- [21] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. <https://qwen.ai/blog?id=qwen3.5>
- [22] Brandon Reagen, Robert Adolf, Yakun Sophia Shao, Gu-Yeon Wei, and David Brooks. 2014. MachSuite: Benchmarks for accelerator design and customized architectures. In *2014 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 110–119.
- [23] RooCodeInc. [n.d.]. Roo-Code. <https://github.com/RooCodeInc/Roo-Code> GitHub repository.
- [24] Nikola Samardžić, Axel Feldmann, Aleksandar Krastev, Srinivas Devadas, Ronald Dreslinski, Christopher Peikert, and Daniel Sanchez. 2021. F1: A fast and programmable accelerator for fully homomorphic encryption. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 238–252.
- [25] Benjamin Carrión Schafer and Zi Wang. 2019. High-level synthesis design space exploration: Past, present, and future. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 10 (2019), 2628–2639.
- [26] Atefeh Sohrabizadeh, Cody Hao Yu, Min Gao, and Jason Cong. 2022. AutoDSE: Enabling software programmers to design efficient FPGA accelerators. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 27, 4 (2022), 1–27.
- [27] Kling Team, Jialu Chen, Yuanzheng Ci, Xiangyu Du, Zipeng Feng, Kun Gai, Sainan Guo, Feng Han, Jingbin He, Kang He, et al. 2025. Kling-Omni Technical Report. *arXiv preprint arXiv:2512.16776* (2025).
- [28] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. J. Stevens, J. Tranchida, C. Trott, and S. J. Plimpton. 2022. LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Comp. Phys. Comm.* 271 (2022), 108171. doi:10.1016/j.cpc.2021.108171
- [29] Hanyu Wang, Xinrui Wu, Zijian Ding, Su Zheng, Chengyue Wang, Tony Nowatzki, Yizhou Sun, and Jason Cong. 2025. LLM-DSE: Searching Accelerator Parameters with LLM Agents. *arXiv preprint arXiv:2505.12188* (2025).
- [30] Jing Wang, Shang Liu, Yao Lu, and Zhiyao Xie. 2025. HLSDebugger: Identification and Correction of Logic Bugs in HLS Code with LLM Solutions. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.

- [31] Chenwei Xiong, Cheng Liu, Huawei Li, and Xiaowei Li. 2024. Hlspilot: Llm-based high-level synthesis. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [32] Kangwei Xu, Grace Li Zhang, Xunzhao Yin, Cheng Zhuo, Ulf Schlichtmann, and Bing Li. 2024. Automated c/c++ program repair for high-level synthesis via large language models. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*. 1–9.
- [33] Kangwei Xu, Grace Li Zhang, Xunzhao Yin, Cheng Zhuo, Ulf Schlichtmann, and Bing Li. 2025. HLSRewriter: Efficient Refactoring and Optimization of C/C++ Code with LLMs for High-Level Synthesis. *ACM Transactions on Design Automation of Electronic Systems* (2025).
- [34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [35] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176* (2025).
- [36] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* 1, 2 (2023), 1–124.